Mapeamento Arquitetural para Git e GitHub

Planejamento da estrutura do repositório do TaskTracker — Preparação para a Fase 2

Nome: Thiago Castro Barreto

Curso: Ciências de Dados

Polo/Turma: Bootcamp 2

Email: thiago.cb@sempreceub.com

Desenho da Árvore de Repositório

Estrutura de pastas e arquivos do TaskTracker que será versionada no GitHub

tasktracker-projeto/ PASTA RAIZ

Arquivos na raiz

README.md
Apresentação do projeto no GitHub

.gitignore
Ignora a pasta __pycache__ do Python

requirements.txt
Dependências (só biblioteca padrão)

Dockerfile
Empacotamento — previsto para a Fase 3

docs/

planejamento_logico.pdf
Documento de planejamento da Fase 1

mapeamento_arquitetural.pdf
Esta apresentação de estrutura

src/

main.py
Ponto de entrada e laço do menu

menu.py
Exibe o menu e lê a opção digitada

tarefas.py
As três funcionalidades do sistema

validacoes.py
Regras de validação das entradas

Observação Importante: *O bloco tracejado não é uma pasta, ele reúne os arquivos que ficam soltos na raiz do repositório, no mesmo nível de docs/ e src/.*

Arquivos da Raiz e Documentação

Finalidade de cada item que fica fora da pasta de código-fonte

- **`README.md`**
  Apresenta o projeto na página inicial do GitHub: o que é o TaskTracker, suas funcionalidades e como executá-lo.

- **`.gitignore`**
  Informa ao Git quais arquivos não devem ir para o repositório, como a pasta __pycache__ gerada pelo Python.

- **`requirements.txt`**
  Lista as dependências do projeto. Nesta fase o TaskTracker usa apenas a biblioteca padrão do Python.

- **`Dockerfile`**
  Receita de empacotamento do programa em container, prevista para a Fase 3 do projeto.

- **`docs/`**
  Reúne a documentação do projeto, mantendo-a separada do código-fonte.

- **`docs/planejamento_logico.pdf`**
  Documento de planejamento lógico da Fase 1, guardado no repositório como registro do projeto.

- **`docs/mapeamento_arquitetural.pdf`**
  Esta apresentação, com o diagrama de hierarquia planejado para o repositório.

- **`src/`**
  Reúne o código-fonte (source) do projeto, seguindo a convenção usada em projetos Python.

Divisão do Código-Fonte em src/

Cada módulo corresponde a uma parte da receita lógica descrita no documento

`main.py` Ponto de entrada do programa

- Cria a lista vazia que guarda as tarefas cadastradas
- Controla o laço principal do menu de opções
- Encerra o programa quando o usuário solicita a saída

`menu.py` Interface com o usuário

- Exibe as opções disponíveis: cadastrar, visualizar, concluir e sair
- Lê a opção digitada e encaminha para a função correspondente
- Avisa o usuário quando a opção digitada é inválida

`tarefas.py` As três funcionalidades

- cadastrar_tarefa() — registra título, descrição, prioridade e data limite (Item 3.1)
- listar_tarefas() — exibe as tarefas ordenadas por prioridade e o resumo estatístico (Item 3.2)
- concluir_tarefa() — altera o status de “Pendente” para “Concluída”

`validacoes.py` Regras de validação das entradas

- Título não pode ficar em branco ou conter apenas espaços (Regra 2.3)
- Prioridade só pode ser Alta, Média ou Baixa (Regra 2.3)
- Tarefa só pode ser concluída se já estiver cadastrada na lista (Regra 2.3)

Plano de Aprendizado para Git e GitHub

O que espero aprender sobre controle de versão na Fase 2 do projeto

01

Commits

Registrar cada alteração de forma organizada, com mensagens claras, para que o histórico conte a evolução do TaskTracker.

02

Branches

Desenvolver e testar cada módulo (main, menu, tarefas, validações) separadamente, sem comprometer a versão que já funciona.

03

Conflitos e histórico

Resolver conflitos de código e navegar pelo histórico para recuperar versões anteriores caso algo quebre.

04

Publicação no GitHub

Publicar o repositório de forma profissional, com README bem escrito e estrutura de pastas coerente.